

Kiosk-Modus fuer die Temperaturueberwachungs-Plattform

Zwei-Benutzer-Architektur auf dem Raspberry Pi 5 (labwc/Wayland)

LoRa-Temperaturueberwachungsprojekt

1. Juli 2026

Inhaltsverzeichnis

1	Zielsetzung	2
1.1	Grundsatz: Server-Hosting und Kiosk-Anzeige sind unabhaengig	2
2	Zwei-Benutzer-Architektur	2
2.1	Kritischer Sicherheitspunkt: Docker-Gruppe	2
3	Einrichtung Schritt fuer Schritt	2
3.1	Schritt 1: Ausgangslage pruefen	2
3.2	Schritt 2: Benutzer kiosk anlegen	3
3.3	Schritt 3: Gruppenmitgliedschaften kontrollieren	3
3.4	Schritt 4: sudo fuer kiosk technisch sperren	3
3.5	Schritt 5: SSH-Zugriff fuer kiosk deaktivieren	3
3.6	Schritt 6: Autologin einrichten (lightdm)	3
3.7	Schritt 7: Konfigurationsverzeichnis fuer kiosk anlegen	4
3.8	Schritt 8: autostart -Datei anlegen (finale Version)	4
3.9	Schritt 9: Ausfuehrbar machen	5
3.10	Schritt 10: Kontrolle	5
4	Login-freier Start des Dashboards	5
4.1	Gewaehlter Weg: Session-Cookie im Kiosk-Profil dauerhaft halten	5
5	Keyring-Problematik und die Entscheidung fuer -password-store=basic	6
5.1	Symptom	6
5.2	Warum es keinen dialogfreien, zugleich verschluesselten Weg gibt	6
5.3	Bewertung fuer dieses Bedrohungsmodell	6
5.4	Verworfen Alternative: Firefox	7
5.5	Umsetzung	7
6	Offene Punkte / Ausblick	7
7	Systemumgebung (Referenz)	8

1 Zielsetzung

Der Raspberry Pi 5 uebernimmt zwei Rollen gleichzeitig: Er hostet den Docker-Compose-Stack (Nginx, Django/Gunicorn, LoRa-Receiver) und dient zusaetzlich als lokales Anzeigeterminal, das das Dashboard permanent im Vollbild zeigt.

Das Sicherheitskonzept ist zweistufig angelegt:

- **Kurzfristig:** Chromium-Kiosk auf dem Pi, mit der Option, lokal per Passwort aus dem Kiosk auszubrechen (Wartung/Debugging in der Aufbauphase).
- **Langfristig:** Vollstaendig gehaertete, ausschliesslich per SSH administrierte Plattform. Der lokale Bildschirm wird zu einem reinen Anzeigegeraet ohne Bedienmoeglichkeit.

1.1 Grundsatz: Server-Hosting und Kiosk-Anzeige sind unabhaengig

Server-Hosting (Docker-Stack) und Kiosk-Anzeige (Chromium) sind funktional vollstaendig getrennt. Sie beruehren sich ausschliesslich ueber `http://localhost`: Der Kiosk-Browser ist aus Sicht des Servers nur ein weiterer Netzwerk-Client. Der Server benoetigt keinen angemeldeten grafischen Benutzer, da der Docker-Daemon als systemd-Hintergrunddienst (*headless*) unabhaengig von jeder Bildschirm-Session laeuft und bereits beim Booten (vor der grafischen Sitzung) startet.

Daraus folgt die Kernentscheidung: zwei getrennte lokale Benutzer mit unterschiedlichen Rollen und Rechten.

2 Zwei-Benutzer-Architektur

Eigenschaft	lorabase (Admin)	kiosk (Anzeige)
Zweck	Server-Administration	Nur Bildschirm-Anzeige
Zugriff	Ausschliesslich SSH	Nur lokaler Autologin
sudo	Ja	Nein (explizit gesperrt)
Docker-Gruppe	Ja	Nein
SSH-Login	Ja	Nein (deaktiviert)
Autologin am Bildschirm	Nein	Ja

2.1 Kritischer Sicherheitspunkt: Docker-Gruppe

Mitgliedschaft in der Gruppe `docker` ist faktisch gleichbedeutend mit `root`-Rechten, da ein Container das gesamte Host-Dateisystem einhaengen kann. Der Benutzer `kiosk` darf daher **niemals** Mitglied dieser Gruppe sein. Andernfalls waere ein Ausbruch aus dem Kiosk gleichbedeutend mit vollem Root-Zugriff.

3 Einrichtung Schritt fuer Schritt

3.1 Schritt 1: Ausgangslage pruefen

```
whoami
groups
getent group docker
```

`getent group docker` listet die Mitglieder der Docker-Gruppe – diese Liste darf `kiosk` spaeter nicht enthalten.

3.2 Schritt 2: Benutzer kiosk anlegen

```
sudo adduser kiosk
```

adduser legt Home-Verzeichnis und Standard-Shell automatisch an. Der Benutzer wird **nicht** zu **sudo** oder **docker** hinzugefuegt. Ein Passwort wird dabei interaktiv vergeben; da der SSH-Login fuer **kiosk** ohnehin deaktiviert wird (Schritt 5), ist dieses Passwort nur fuer den lokalen Zugriff relevant.

3.3 Schritt 3: Gruppenmitgliedschaften kontrollieren

```
groups kiosk
```

Erwartet: nur die eigene Gruppe **kiosk** sowie ggf. **video/render/input** (fuer Bildschirm- und Touch-Zugriff). Falls **sudo** oder **docker** erscheint:

```
sudo deluser kiosk sudo
sudo deluser kiosk docker
```

3.4 Schritt 4: sudo fuer kiosk technisch sperren

```
sudo visudo -f /etc/sudoers.d/kiosk-no-sudo
```

Inhalt:

```
kiosk ALL=(ALL) !ALL
```

visudo prueft die Syntax vor dem Speichern und verhindert damit, dass ein Fehler die sudoers-Konfiguration beschaedigt.

3.5 Schritt 5: SSH-Zugriff fuer kiosk deaktivieren

```
sudo nano /etc/ssh/sshd_config
```

Am Ende der Datei ergaenzen:

```
Match User kiosk
    PasswordAuthentication no
    PubkeyAuthentication no
```

Danach:

```
sudo systemctl reload ssh
```

kiosk kann sich damit unter keinen Umstaenden per SSH anmelden – unabhaengig vom gewaehlten Passwort. SSH-Scans aus dem Netz laufen fuer diesen Benutzernamen bereits auf Protokollebene ins Leere.

3.6 Schritt 6: Autologin einrichten (lightdm)

Auf diesem System ist **lightdm** als Display-Manager aktiv (nicht **greetd**). Konfigurationsdatei:

```
sudo nano /etc/lightdm/lightdm.conf
```

Wichtige Falle: Die Option `autologin-user` muss innerhalb des Abschnitts `[Seat:*)` stehen, nicht im vorangehenden `[LightDM]`-Abschnitt. In `.conf`-Dateien gehoert jede Zeile zum darueberstehenden Abschnitts-Header; eine im falschen Abschnitt platzierte Zeile wird stillschweigend ignoriert.

Korrekte Platzierung innerhalb von `[Seat:*)`:

```
[Seat:*)
...
autologin-user=kiosk
autologin-session=rpdr-labwc
```

Ein evtl. bereits vorhandener Wert (z. B. `autologin-user=lorabase`) wird auf `kiosk` geaendert. Die Sitzungsart (`autologin-session=rpdr-labwc`) bleibt unveraendert.

Anwenden und testen – ein vollstaendiger Neustart ist zuverlaessiger als ein reiner Dienst-Neustart, da alte Sitzungen sonst als Zombie-Rest bestehen bleiben koennen:

```
sudo reboot
```

Kontrolle der aktiven Sitzung:

```
who
loginctl list-sessions
```

Erwartet: `kiosk` aktiv auf `seat0`; `lorabase` nur noch ueber eine SSH-Sitzung (`sshd pts/0`), nicht mehr auf `seat0/tty1`.

SSH-Zugriff nach der Umstellung erfolgt unveraendert ueber:

```
ssh lorabase@<IP-des-Pi>
```

Grafischer Autologin (`kiosk`) und SSH-Zugang (`lorabase`) sind vollstaendig unabhaengige Zugriffswege.

3.7 Schritt 7: Konfigurationsverzeichnis fuer kiosk anlegen

```
sudo -u kiosk mkdir -p /home/kiosk/.config/labwc
```

`sudo -u kiosk` (statt einfachem `sudo`) stellt sicher, dass das Verzeichnis `kiosk` gehoert und nicht `root`.

3.8 Schritt 8: autostart-Datei anlegen (finale Version)

```
sudo -u kiosk tee /home/kiosk/.config/labwc/autostart > /dev/null << 'EOF'
#!/bin/bash
chromium \
  --kiosk \
  --ozone-platform=wayland \
  --noerrdialogs \
  --disable-infobars \
  --no-first-run \
```

```
--disable-pinch \  
--overscroll-history-navigation=0 \  
--password-store=basic \  
http://localhost &  
EOF
```

Erlaeuterung der Chromium-Optionen:

- `-kiosk` – Vollbild ohne Adressleiste, Tabs, Menues.
- `-ozone-platform=wayland` – nutzt explizit das Wayland-Protokoll (korrekt fuer labwc auf dem Pi 5).
- `-noerrdialogs` – unterdrueckt Chromium-Fehlerfenster.
- `-disable-infobars` – verhindert Hinweisleisten.
- `-no-first-run` – ueberspringt den Einrichtungsassistenten.
- `-disable-pinch`, `-overscroll-history-navigation=0` – verhindern versehentliches Zoomen bzw. Wisch-Navigation bei Touch-Bedienung.
- `-password-store=basic` – siehe Abschnitt 5; verhindert den gnome-keyring-Unlock-Dialog.
- `http://localhost &` – Ziel-URL; & fuehrt Chromium im Hintergrund aus.

Voraussetzung: Der Docker-Compose-Stack muss zu diesem Zeitpunkt bereits auf Port 80 antworten. Bei abweichendem Port entsprechend `http://localhost:<port>`.

3.9 Schritt 9: Ausfuehrbar machen

```
sudo -u kiosk chmod +x /home/kiosk/.config/labwc/autostart
```

3.10 Schritt 10: Kontrolle

Da `/home/kiosk` mit Rechten `0700` angelegt wird, kann `lorabase` den Inhalt nicht direkt einsehen (Permission denied ist hier erwartetes Verhalten). Einsicht ueber:

```
sudo cat /home/kiosk/.config/labwc/autostart  
sudo ls -l /home/kiosk/.config/labwc/autostart
```

4 Login-freier Start des Dashboards

Da die Anwendung durch ein geteiltes Passwort (`RequireLoginMiddleware`) geschuetzt ist, wuerde der Kiosk beim Start zunaechst das Login-Formular zeigen. Ziel: nur der Kiosk-Bildschirm startet ohne Login, alle anderen Zugriffe im LAN bleiben unveraendert geschuetzt.

4.1 Gewaehlter Weg: Session-Cookie im Kiosk-Profil dauerhaft halten

Der Kiosk wird *einmalig* manuell eingeloggt. Django setzt ein Session-Cookie, das im persistenten Chromium-Profil (`/home/kiosk/.config/chromium`) gespeichert wird und Neustarts uebersteht. Der Login-Schutz fuer alle anderen Clients bleibt vollstaendig unberuehrt, da diese Loesung nur dieses eine Chromium-Profil betrifft.

Django-Einstellungen (`settings.py`), damit die Session einen Neustart uebersteht:

```
SESSION_EXPIRE_AT_BROWSER_CLOSE = False
SESSION_COOKIE_AGE = 60 * 60 * 24 * 365 # 1 Jahr in Sekunden
```

Wichtig: Der `SECRET_KEY` signiert die Session-Cookies. Aendert er sich bei einem Container-Neustart, werden alle Sessions ungultig. Er muss daher fest (z. B. in der `.env` bzw. `docker-compose.yml`) gesetzt sein und nicht bei jedem Start neu generiert werden.

Einmaliger Login: `autostart` temporaer ohne `-kiosk` starten (normales Fenster mit Adressleiste), am Bildschirm einloggen, danach die `autostart`-Datei zurueck auf die Kiosk-Version (Schritt 8) setzen.

5 Keyring-Problematik und die Entscheidung fuer `-password-store=basic`

5.1 Symptom

Beim ersten Login im Kiosk-Browser erscheint ein Dialog *"Choose new keyring"* bzw. *"Unlock keyring"*. Ursache: Chromium legt seinen internen Verschluesselungsschluessel (OS-Crypt, fuer Cookies) im System-Schluesselbund (`gnome-keyring`) ab. Auf einer schlanken labwc-Installation ist dieser Keyring nicht vorkonfiguriert; er verlangt daher das Anlegen bzw. Entsperren mit einem Master-Passwort.

5.2 Warum es keinen dialogfreien, zugleich verschluesselten Weg gibt

Der Keyring wird ueber PAM mit dem Login-Passwort entsperrt. Bei einem **Autologin** tippt niemand ein Passwort ein, folglich kann PAM dem Keyring kein Passwort weiterreichen. Automatisches Entsperren zusammen mit Autologin ist nur mit einem **leeren Keyring-Passwort** moeglich – dann liegt der Inhalt aber ebenfalls unverschlueselt vor.

Konsequenz: Solange die Maschine ohne Passworтеingabe hochfaehrt und sich selbst einloggt, ist jeder lokal gespeicherte Schluessel zwangslaeufig durch etwas geschuetzt, das ebenfalls auf der Maschine liegt. Ein leeres Keyring-Passwort, `-password-store=basic` und das Standardverhalten von Firefox (ohne Primary Password) sind in diesem Punkt sicherheitstechnisch gleichwertig – der Schluessel ist jeweils fuer jeden lesbar, der Zugriff auf das Dateisystem hat.

Echte Verschlueselung der Schluessel *at rest* bei erhaltenem Autologin ist nur eine Ebene tiefer erreichbar: Full-Disk-Encryption (LUKS) mit Boot-Passwort oder TPM. Ein Boot-Passwort bricht jedoch den unbeaufsichtigten Autostart, und der Pi 5 hat keinen Standard-TPM.

5.3 Bewertung fuer dieses Bedrohungsmodell

Auf dem `kiosk`-Benutzer werden keine sensiblen Passwoerter gespeichert (Passwort-Manager per Policy deaktiviert). Das einzige schuetzenswerte Objekt waere das Django-Session-Cookie, das jedoch nur Zugriff auf ein Dashboard gewaehrt, das im selben LAN mit demselben geteilten Passwort ohnehin erreichbar ist. Der Keyring schuetzt Benutzer voreinander bzw. gegen Datentraeger-Diebstahl im Ruhezustand – beides sind fuer einen Single-Purpose-Kiosk mit nur einem unprivilegierten Anzeige-Benutzer keine relevanten Szenarien.

Die tragende Sicherheit dieses Setups liegt nicht im Keyring, sondern in: keiner Docker-Gruppe fuer `kiosk`, keinem `sudo`, deaktiviertem SSH-Login und der SSH-Haertung des Admin-Kontos. An diesen Punkten aendert `-password-store=basic` nichts.

5.4 Verwerfene Alternative: Firefox

Ein Wechsel zu Firefox wurde erwogen und wieder verworfen. Firefox vermeidet den Keyring-Dialog zwar (eigenes verschluesseltes Profil statt System-Keyring), bietet aber ohne Primary Password kein hoeheres Schutzniveau fuer die Cookies als `-password-store=basic`. Zudem trat unter labwc/Wayland ein Startproblem auf, und auf dem System heisst das Paket `firefox` (nicht `firefox-esr`), was den urspruenglichen Aufruf ins Leere laufen liess. Da die Chromium-Loesung funktioniert und alle uebrigen System-Schritte Chromium-unabhaengig bestehen bleiben, wurde bei Chromium geblieben.

5.5 Umsetzung

Flag in autostart (siehe Schritt 8): `-password-store=basic`.

Zusaetzlich eine Chromium-Enterprise-Policy, um die "Passwort speichern?"- und Uebersetzungs-Dialoge zu unterbinden (Paketname `chromium`, daher Pfad `/etc/chromium/policies/managed/`):

```
sudo mkdir -p /etc/chromium/policies/managed
sudo tee /etc/chromium/policies/managed/kiosk_policy.json > /dev/null << 'EOF'
{
  "PasswordManagerEnabled": false,
  "TranslateEnabled": false
}
EOF
```

Eventuelle Reste frueherer Versuche entfernen:

```
sudo rm -rf /home/kiosk/.local/share/keyrings
```

Kontrolle im Nicht-Kiosk-Fenster: `chromium://version` zeigt in der Kommandozeile `-password-store=basic`; `chromium://policy` listet die aktiven Richtlinien.

6 Offene Punkte / Ausblick

- **Boot-Reihenfolge:** Startet Chromium schneller als der Docker-Stack bereit ist, erscheint kurz *"Seite nicht erreichbar"*. Loesung: Warte-Skript, das vor dem Chromium-Start auf den Port pollt, oder automatisches Neuladen bei Ladefehler.
- **Passwort-Ausbruch aus dem Kiosk:** Chromium bietet keinen nativen Passwort-Dialog zum Verlassen des Kiosk-Modus. Fuer die Uebergangsphase geplant: versteckte Tastenkombination in `/.config/labwc/rc.xml`, die ein Skript mit `zenity -password`-Abfrage ausloest und Chromium erst nach korrektem Passwort beendet (`pkill chromium`).
- **Langfristige Haertung:** Perspektivisch Umstieg auf einen dedizierten Minimal-Compositor (`cage`), der nur ein Vollbild-Fenster zeigt und keine Desktop-Umgebung bereitstellt, aus der ausgebrochen werden koennte. Kiosk als `systemd`-Service, steuerbar per SSH.
- **SSH-Haertung des Admin-Kontos (lorabase):** Pruefung, ob Port 22 nur im LAN oder auch aus dem Internet erreichbar ist; Umstellung auf reine Key-Authentifizierung; optional `fail2ban`.
- **Session-Sicherheit im Transport:** Fuer das langfristige Ziel ggf. HTTPS im LAN, damit Session-Cookies auch auf dem Uebertragungsweg geschuetzt sind (relevanter fuer Session-Sicherheit als die lokale Keyring-Frage).

7 Systemumgebung (Referenz)

- Raspberry Pi 5, Debian 13 (Trixie)-basiertes Raspberry Pi OS, Kernel 6.12, RP1-GPIO.
- Display-Stack: Wayland mit `labwc` als Compositor, `lightdm` als Display-Manager.
- Browser: Paket `chromium` (nicht `chromium-browser`), Version 149, mit `rpi-chromium-mods`.
- Anwendung: Django hinter Nginx/Gunicorn im Docker-Compose-Stack, Zugriff ueber `http://localhost`.